# PPT-LAT-S: A Decentralized Cooperative Network for Crawling, Training, and Inference on the Prime-Factored Lattice

**Companion paper to PPT-LAT-T (theory). Cite as PPT-LAT-S §N.**

---


## Abstract

We describe **shannon-prime-lattice**, a decentralized cooperative network in which volunteer nodes contribute CPU/GPU time to crawl the public web, train shared model weights, and serve inference, all coordinated through a single algebraic substrate: the prime-factored coordinate lattice introduced in the companion theory paper (PPT-LAT-T). Three primitives recur at every layer of the stack: (i) KSTE encoding of content into 64-byte packed trees; (ii) Friedman-sieve dominance (≤_d) as the universal admit/reject decision; and (iii) the CRT-decomposed cyclotomic ring Z_q[x]/(x^N+1) as both the inference engine and the gossip substrate via HRR bindings.

The network's central design bet is that **the same lattice geometry that makes attention tractable also makes coordination tractable**. URLs hash to lattice coordinates so that semantic adjacency is spatial adjacency; nodes own slabs of the lattice and therefore inherit a natural sharding of both data and compute; gradient updates aggregate via HRR circular convolution which is automatic in capacity decay and commutative under gossip; inference splits along the CRT residues so that two nodes computing mod q1 and mod q2 can combine their results with no precision loss and kilobyte-scale wire traffic. Verification is asymmetric in the Bitcoin sense — doing requires the full KSTE/ARM/CRT pipeline; checking requires only a primitive-recursive ≤_d embedding decision (PPT-LAT-T §3). A two-token economy (Work / Discovery) compensates effort and rewards dominance-novel contributions to the public frontier.

This paper specifies the six architectural layers, the wire formats, the epoch and slashing structure, and the failure modes we currently believe the system can and cannot survive. We are honest at the end about what this paper does not establish: chiefly, the scaling constants and the adversarial economic equilibria, both of which require deployment evidence we do not yet have.


---

## 1. Introduction and design goals

### 1.1 What this system is for

The dominant trend in large-model AI is centralization of three resources: crawl corpora, training compute, and inference serving. Each is held by a small number of operators with strong incentives not to share. Decentralized alternatives exist (BitTorrent, Filecoin, BOINC, Hivemind, Petals, Bittensor) but each addresses one resource in isolation and accepts a heavy coordination cost (proof-of-work, model parallelism over commodity internet, on-chain settlement of every microtask).

The bet of shannon-prime-lattice is that a single algebraic structure — the prime-factored lattice with its KSTE/≤_d/CRT/ARM primitives — collapses the coordination cost across all three resources simultaneously. Crawl assignment becomes lattice slab ownership. Training aggregation becomes ring addition. Inference sharding becomes CRT recombination. Verification becomes dominance embedding. Token economics becomes dominance-novelty minting.

The user-facing claim is modest: a volunteer node with a consumer GPU and a residential internet connection should be able to (a) crawl and KSTE-encode a slab of the web, (b) participate

in training a shared model whose weights live partly in its slab, and (c) serve a fraction of an inference request, all within the same daemon, with the same wire format, and with the same verification primitive.

### 1.2 Non-goals

We are not building: - A general-purpose blockchain. The ledger is a frontier of dominance-incomparable KSTE commitments, not a transaction log. - A privacy-preserving system. Raw page text never leaves the host (because only the KSTE tree is published), but this is *publishing minimization*, not anonymity. Adversaries can correlate lattice coordinates with topic clusters. - A guaranteed-uptime inference service. CRT sharding degrades gracefully under partition; it does not eliminate partition. - A replacement for centralized training of frontier models. We expect the network to be competitive at the long-tail end (small-to-mid models, many fine-tunes, broad crawl coverage) and uncompetitive at the head (a single 1T-parameter dense model).

### 1.3 Document structure

Sections 2–7 describe the six architectural layers. Section 8 specifies wire formats and protocol parameters. Section 9 covers bootstrap, epochs, and governance. Section 10 is the failure modes / adversarial section. Section 11 compares to prior work. Section 12 enumerates what this paper does not establish.

Throughout, references of the form "PPT-LAT-T §N" point to the companion theory paper. We do not re-derive the math; we cite and build on it.

---

## 2. Layer 1 — Crawler and knowledge representation

### 2.1 Lattice-keyed DHT

The network's routing substrate is a Kademlia-style DHT in which the key space is **not** the usual uniform 160-bit hash output but the **prime-factored coordinate lattice** L of PPT-LAT-T §2. A URL u is mapped to a coordinate `c(u) ∈ L` by a deterministic hash that factors the URL into (registrable_domain, path_segments, query_terms, content_type) and emits a tuple of small-prime exponents:

```
c(u) = (e_2, e_3, e_5, e_7, ..., e_p_k)
```

where each `e_i` is determined by hashing a different component of u modulo a small bound. Adjacent URLs — same domain, sibling paths — share most coordinates and differ in only one or two exponents. The resulting key space inherits the lattice's order structure: the partial order `c(u) ≤ c(v)` corresponds to "u is a coordinate-wise ancestor of v," which approximates semantic ancestry in domain/topic space.

The crucial consequence is that **routing distance in the DHT corresponds to semantic distance**. A node responsible for a slab of L is not responsible for a random pseudo-uniform slice of URL space; it is responsible for a contiguous semantic neighborhood. This collapses the gap between content-addressed and topic-addressed storage that has dogged prior DHT designs (Coral, OpenDHT, IPFS).

Routing itself proceeds as in Kademlia with the XOR metric replaced by the lattice's L1 distance on exponent vectors. Iterative lookup converges in O(log |network|) hops because the lattice is balanced under the chosen prime alphabet (PPT-LAT-T §2.4).

### 2.2 Slab ownership

A node n advertises a **slab** S_n ⊂ L — a bounded region of coordinates. Slabs may overlap; overlap implies replication. A page u with coordinate c(u) ∈ S_n is n's responsibility: n is the authoritative crawler, KSTE-encoder, and cache holder for that page.

Slab size is bargained at join time (Section 9.1). A small node may claim a single coordinate; a large node may claim a slab covering 10^6 coordinates. The network maintains a coverage invariant (every coordinate is in at least r slabs, where r is the replication factor, default r = 3) by reassigning slabs when nodes leave or join. Reassignment cost is bounded because the lattice partial order makes hand-off contiguous: a leaving node hands its slab to neighbors who already hold adjacent regions and therefore already have most of the relevant cache.

### 2.3 KSTE encoding and the local cache

For each page in its slab, a node performs **KSTE encoding** (PPT-LAT-T §4): a deterministic, content-defined tree extraction producing a 64-byte packed representation. The encoding is purely local; raw text need never leave the host. The packed tree is the unit of:

- Storage in the local cache.
- Wire transmission to peers.
- Commitment in the verification protocol (Layer 5).

The local cache is **bounded by semantic novelty, not crawl volume**, via the **Friedman sieve**: a candidate tree T_new is admitted to the cache iff there is no T_old already in the cache with T_new ⊴_d T_old. Equivalently, the cache stores the dominance-incomparable frontier of trees that the node has encountered. By the wqo property (PPT-LAT-T §3.2), this frontier is finite for any bounded coordinate domain — so the cache size grows sub-linearly in crawl volume and eventually saturates.

Pseudocode for the admit decision:

```
function admit(T_new, cache):
    for T_old in cache.iter_neighbors(T_new):
        if dominance_embed(T_new, T_old):
            return REJECT_SUBSUMED
        if dominance_embed(T_old, T_new):
            cache.evict(T_old)        # T_new dominates T_old
    cache.insert(T_new)
    return ADMIT_NOVEL
```

`cache.iter_neighbors` uses the lattice coordinate of the page to fetch only the spatially adjacent trees — most trees in the cache are irrelevant to the embedding check and need not be scanned. This keeps the admit decision O(neighborhood size) regardless of cache size.

### 2.4 Re-crawl scheduling

A coordinate's freshness decays at a rate inferred from the page's update history and its lattice position. Coordinates in fast-moving slabs (news, forums) decay quickly; coordinates in slow-moving slabs (archives, reference) decay slowly. The decay function is itself a small KSTE-shaped expression evaluated at the coordinate, so it inherits the lattice geometry and stays cheap to compute.

---

## 3. Layer 2 — Cross-node aggregation via ARM (HRR over the cyclotomic ring)

### 3.1 The local HRR state

Every node maintains a local **Algebraic Resonance Memory (ARM)** state — a single element `s_n ∈ R_q := Z_q[x]/(x^N+1)`, with `N` a power of two and `q` a product of CRT-compatible Proth primes. The state represents the node's accumulated contribution to the shared model: gradient updates, weight deltas, fine-tune signals.

Each contribution is bound into the state by **circular convolution with a key**:

```
s_n ← s_n + k ⊛ v
```

where k is a key derived from (epoch, lattice_coordinate, contribution_kind) and v is the value to be stored (the gradient delta, packed and projected into R_q). Circular convolution is HRR's binding operator (Plate 1995); in our setting it is a single negacyclic polynomial multiplication, which is cheap because N is moderate (we use N=256 by default) and the multiplication runs through the CRT NTT kernel from PPT-LAT-T §5.

### 3.2 Gossip aggregation

Periodically (every gossip round, default 30 s), nodes exchange compressed states with their lattice neighbors. Merging is **addition in R_q**:

```
s_merged = s_a + s_b   (mod q, coefficient-wise)
```

This operation is associative and commutative, so the order in which gossip propagates does not affect the result. Two nodes that have seen the same set of contributions in different orders converge to the same state. This is the property that lets us avoid Paxos/Raft-style consensus: ARM states are CRDTs (PPT-LAT-T §6.1 establishes the CRDT property formally).

### 3.3 Capacity decay is automatic

HRR has a built-in capacity bound: as more pairs (k_i, v_i) are bound into a state, recall fidelity per pair degrades smoothly. This is normally a problem; in our setting it is a feature. Old contributions fade as new ones accumulate, with no explicit timeout or LRU policy. The decay rate is controlled by the gossip frequency and the binding rate — no hyperparameter.

When the network observes that a region of the lattice is heavily contributed-to (e.g., a popular topic during a news event), the local ARM states in that region saturate. The next layer (slab subdivision) is triggered: the saturated slab splits, and each child node inherits a sub-ring `Z_q[x]/(x^{N/2}+1)` carrying half the coefficients. This is the **ring decomposition** mechanism (PPT-LAT-T §6.2).

### 3.4 Recall and gradient assembly

When a node needs to recover an aggregated gradient at coordinate c, it queries its ARM state with the same key `k(epoch, c, kind)` used for binding:

```
v_recall = k* ⊛ s_n
```

where k* is the involutive inverse of k (negacyclic involution, PPT-LAT-T §4.3). The recall is approximate — capacity decay means older contributions are noisier — but the noise is small and zero-mean, and downstream gradient application tolerates it the same way SGD tolerates mini-batch noise.

### 3.5 Failure modes (in-line preview)

Three failure modes are specific to Layer 2 and worth flagging here; we treat them again in Section 10.

- **Byzantine bindings**: a malicious node binds garbage into a state. Caught by Layer 5: the binding's KSTE commitment fails ≤_d verification against the published epoch frontier.
- **Capacity exhaustion under heavy aggregation**: handled by ring decomposition (§3.3) plus epoch rotation (§9.2).
- **Stale gradients**: handled by automatic decay (§3.3) — no explicit staleness penalty needed.

---

## 4. Layer 3 — Inference sharding via CRT NTT

### 4.1 The CRT split

The expensive primitive inside our attention layers is a polynomial multiplication in `R_q = Z_q[x]/(x^N+1)`. By construction, `q = q_1 · q_2 · ... · q_k` where each `q_i` is a 30-bit Proth prime with the right roots-of-unity properties for NTT. The Chinese Remainder Theorem says the ring decomposes:

```
R_q  ≅  R_{q_1} × R_{q_2} × ... × R_{q_k}
```

A polynomial multiplication in R_q is equivalent to k independent multiplications in the residue rings `R_{q_i}`, plus a CRT recombination of the results. The recombination is a fixed linear map and costs O(N) per attention head.

### 4.2 Inference dispatch

For a two-prime split (k=2, our default), an inference request flows as follows:

1. **Driver node** (the node closest to the request originator in the lattice) receives the prompt, computes its lattice coordinate, and identifies the two CRT shards responsible for the relevant model partition.
2. Driver sends the prompt embedding (a single polynomial in R_q) to **shard A** as residues mod q_1, and to **shard B** as residues mod q_2. Wire size per token is `N · log_2(q_i) / 8` bytes — kilobytes, not megabytes.
3. Shard A and shard B independently compute their attention layers mod q_1 and mod q_2.
4. At each layer boundary (or at the end of the forward pass, depending on schedule), shards return residues to the driver.
5. Driver CRT-recombines and forwards to the next layer.

Crucially, the residue streams are **bit-exact reconstructions** of the full-precision computation — there is no rounding loss in the CRT step (this is the property that made the Phase 9 CRT NTT work end-to-end in earlier shannon-prime engine measurements; we cite this as engineering experience, not as a theoretical claim of this paper).

### 4.3 Verification by recomputation

The driver periodically (default: 1 in 1024 layer dispatches) selects a layer boundary, asks shard A to also compute mod q_2, and checks byte-equality with shard B's output. The cost is one extra forward pass per 1024, distributed across shards; the safety is that any divergence is immediately detected and the offending shard's stake is slashed (Section 9.3).

### 4.4 Larger sharding for resilience

k > 2 trades latency (more network round trips) for resilience: any k-1 of k shards can produce a verifiable lower-precision result by working in `R_q / R_{q_i}` for the missing prime. This is the **graceful degradation** property: a network partition that disconnects one shard does not stop inference; it merely lowers the working modulus.

For k=4 (a typical production setting) the working modulus after losing one prime is still ~90 bits, well within the precision budget of any practical model's attention.

### 4.5 Pseudocode for the driver loop

```
function inference_step(prompt_token, model_id, k=2):
    c = lattice_coord(prompt_token, model_id)
    shards = network.locate_shards(c, k)         # k nodes
    embed = embedding_lookup(prompt_token)        # element of R_q
    residues = [embed mod q_i for q_i in PRIMES]
    for layer in model.layers:
        # parallel dispatch
       outs = parallel([shards[i].apply(layer, residues[i]) for i in range(k)])
        residues = outs
        if epoch.is_verification_step():
            verify_one_shard_against_another(shards, layer, residues)
    return crt_recombine(residues)
```

---

## 5. Layer 4 — Position-as-arithmetic network geometry

### 5.1 One lattice, two purposes

The prime-factored coordinate lattice serves both **sequence position inside attention** (its original purpose in PPT-LAT-T §2) and **node position in the network** (this paper). The two uses are not analogies; they are the same structure used at different scales. The lattice that gives attention its O(N log N) reach also gives the network its O(log N) routing.

### 5.2 Routing is arithmetic

A node looking up the holder of coordinate c does not perform a generic DHT lookup. It performs **coordinate arithmetic**: starting from its own coordinate `c_self`, compute the path to c as a sequence of small-prime steps. Each step moves to a neighbor in the lattice; each step is a single hop in the DHT overlay. The number of steps is the L1 distance between coordinates, which is logarithmic in network size for the chosen prime alphabet.

The practical consequence is that routing tables are small (each node knows the O(log N) neighbors along each prime axis) and lookups are predictable (no random pseudo-uniform hashing).

### 5.3 Load balancing via discovery incentive

Hot regions of the lattice — topics undergoing rapid change, popular inference targets — pull more nodes via the **discovery-token incentive** (Layer 6). Specifically, the discovery token mints faster in regions where the dominance frontier is still growing, which is precisely the regions with high crawl/training activity. Nodes seeking discovery tokens migrate into those regions, increasing replication and reducing per-node load.

Conversely, dormant regions (settled archives, stable reference material) saturate their frontier and stop minting discovery tokens. Nodes there earn only work tokens for serving inference, and the resulting equilibrium concentrates compute where it is needed.

This is a self-tuning load balancer with no central coordinator; it relies entirely on the mathematical property that the dominance frontier grows fastest where the underlying content stream is most novel.

### 5.4 Freshness decay encoded in coordinates

Lattice coordinates encode their own freshness expectations (§2.4). A node scheduling recrawls walks its slab in coordinate order, prioritizing high-decay coordinates. No external scheduler is needed.

---

## 6. Layer 5 — Verification via Friedman-sieve dominance

### 6.1 Every contribution carries a KSTE commitment

Whatever a node claims to have done — crawled a page, bound a gradient update, computed an inference residue — it publishes a **KSTE-encoded commitment** of the result. The commitment is 64 bytes (the packed tree); the verification is a dominance embedding check.

Pseudocode for the verifier:

```
function verify(claim):
    T_claimed   = claim.commitment
    T_published = epoch.frontier[claim.coordinate]
    return dominance_embed(T_claimed, T_published)
```

By PPT-LAT-T §3, `dominance_embed` is **primitive recursive** in the size of the trees. With our 64-byte packed format, the check is microseconds.

### 6.2 The Bitcoin asymmetry

Producing a valid contribution requires running the actual crawl / training / inference pipeline. Checking a contribution requires only the dominance embedding. This is the same asymmetry that makes Bitcoin's proof-of-work tractable, except that here:

- **The work is useful** (it produces a crawled page, a trained gradient, an inference result), not a brute-force hash search.
- **The verification is exact, not probabilistic** — a false negative is mathematically impossible by the wqo property (PPT-LAT-T §3.2).
- **The difficulty is set by the dominance frontier**, which is self-adjusting: as the frontier grows, novel contributions become rarer and the discovery token's mint rate naturally slows.

### 6.3 False-positive analysis

A "false positive" in our setting means the verifier accepts a contribution that should have been rejected. This can happen if a tree T_attacker happens to embed $\leq_d$ into the published frontier despite not being a faithful encoding of any real content. Two arguments why this is benign:

- **Over-inclusion costs the attacker, not the network**: the attacker has done real work to produce a tree that embeds. If the work was the actual encoding of garbage, the

garbage is provably ≤_d-comparable with existing entries, so it adds nothing novel and earns no discovery token.
- **The dominance frontier is public**: any node can audit and propose eviction of frontier entries whose downstream usage (e.g., inference success rate, training contribution) is degenerate.

False negatives are impossible: if the contribution genuinely extends the frontier, ≤_d will say so.

### 6.4 Dominance proofs

A **dominance proof** is the short transcript showing the embedding witness — at most O(tree_size) tree-node identifiers. We publish these proofs alongside the commitments so any third party can re-verify without re-running the embedding algorithm.

---

## 7. Layer 6 — Two-token economy

### 7.1 Two tokens, two roles

The economy uses two tokens with deliberately different inflation curves:

| Token | Mints on | Inflation | Role |
|---|---|---|---|
| **Work Token (WRK)** | Verified compute contribution (crawl / binding / inference branch) | High, linear with verified work | Fee currency for inference requests |
| **Discovery Token (DSC)** | Dominance-incomparable contribution to the frontier | Bounded by wqo property; deflationary in mature regions | Governance + staking weight |

The dual structure is the design's central economic claim. Single-token compute coins (Filecoin, Bittensor at various points) tend toward a fee-token-to-zero collapse: the token's only sink is fees, the supply grows linearly with compute, and once speculation cools the price floors. Adding a second token with **deflationary** properties (DSC, bounded by the wqo) gives long-term holders a separate asset whose supply curve does not depend on compute volume.

### 7.2 WRK mint conditions

WRK mints when: - A crawled page passes Layer 5 verification: the crawler earns WRK proportional to the page's lattice-weighted size. - A gradient binding is accepted into the next epoch's frontier: the binder earns WRK proportional to the binding's compute cost (measured in NTT operations). - An inference branch returns a residue that passes either the per-batch verification recomputation (§4.3) or a downstream user's acceptance signal: the shard earns WRK proportional to the layer's FLOP count.

### 7.3 DSC mint conditions

DSC mints **only** when a contribution extends the public dominance frontier — i.e., the contribution is not ≤_d-subsumed by any existing frontier element, and the frontier has not already saturated at that lattice coordinate.

The mathematical consequence is that DSC is **asymptotically deflationary**: as the network matures and the frontier approaches saturation at each coordinate, DSC mint rate at that coordinate falls to zero. New mint opportunities arise only in three places: (i) genuinely new content on the web, (ii) new lattice coordinates opened up by model architecture changes (e.g., a new modality), and (iii) recovery of evicted frontier elements after a re-organization.

### 7.4 Economic flows

```
Users -> inference fees in WRK -> shard nodes
Crawlers -> WRK for verified crawls
Trainers -> WRK for accepted bindings + DSC for novel bindings
DSC holders -> governance votes + staking yield in WRK
```

Slashing: bad behavior (Section 9.3) burns staked DSC and forfeits unrealized WRK.

### 7.5 Why this avoids the work-token-to-zero collapse

The collapse pattern in prior compute coins is: 1. Initial high inflation rewards early miners. 2. Speculative demand absorbs the inflation. 3. Real demand (fees) never catches up to speculative supply. 4. Price falls to marginal cost of compute, which is zero on amortized hardware. 5. Compute exits, network dies.

DSC breaks the chain at step 4. As the network matures, DSC supply growth asymptotes (wqo). DSC accrues a governance premium and a staking yield denominated in WRK. WRK demand from inference fees is real and growing as long as the model is useful. The two-asset structure means speculation parks in DSC (deflationary) while WRK clears the fee market — and DSC's existence underwrites WRK's value by giving compute providers a path to long-term equity in the network, not just a short-term fee stream.

We are not making a guarantee that this avoids the collapse; we are describing the design intent. Section 12 lists the open empirical questions.

---

## 8. Protocol details and wire formats

### 8.1 Wire format for KSTE commitments

A KSTE commitment on the wire is exactly **64 bytes**:

```
struct kste_commitment {
    uint8  version;            // 1 byte
    uint8  tree_arity;         // 1 byte (small, bounded)
    uint16 leaf_count;         // 2 bytes
    uint32 coord_low;          // 4 bytes  (low 32 bits of lattice coord hash)
    uint64 coord_high;         // 8 bytes  (high 64 bits)
    uint8  packed_tree[48];    // 48 bytes (the actual tree encoding)
};
```

The 48-byte packed tree uses the dense exponent representation described in PPT-LAT-T §4.2. We do not re-derive the format here.

### 8.2 Wire format for HRR state gossip

HRR states are large (N=256 coefficients × log_2(q) bits each ≈ 1.9 KB per residue, ≈ 3.8 KB for k=2 primes). To make gossip cheap, nodes exchange **state deltas** since the last gossip round, not full states:

```
struct hrr_delta {
    uint32 epoch;
    uint32 sender_id;
    uint32 receiver_hint;       // lattice-coord of intended neighbor
    uint16 binding_count;       // number of new bindings in this delta
    uint16 prime_id;            // which CRT residue (0..k-1)
    int32  coefficients[N];     // N=256 int32 (signed residues mod q_i)
    uint8  signature[64];       // Ed25519 over the above
};
```

A typical gossip round between two neighbors is one delta per CRT residue, so ~2 KB × k for k=2 primes ≈ 4 KB per round per neighbor. Each node has O(log N) lattice neighbors (Section 5.2), so total gossip bandwidth is bounded by O(log N) × 4 KB per round.

### 8.3 Wire format for CRT residue exchange

Inference dispatch sends one residue stream per CRT prime to each shard. Per-layer residue:

```
struct crt_layer_residue {
    uint32  epoch;
    uint32  request_id;
    uint16  layer_index;
    uint16  prime_id;
    int32   residues[N * heads];  // N=256 coefficients, per head
    uint8   shard_signature[64];
};
```

For our default Gemma3-class model with N=256, 4 heads, k=2 primes, per-layer residue is ~8 KB; full forward pass over (say) 32 layers is ~256 KB per shard per inference step.

### 8.4 Wire format for dominance proofs

A dominance proof is a sequence of (node_id_in_T_claimed, node_id_in_T_published) pairs encoding the embedding witness:

```
struct dominance_proof {
    uint32 claim_id;
    uint16 pair_count;
    struct { uint16 a; uint16 b; } pairs[pair_count];
};
```

For our 64-byte trees with bounded arity, `pair_count` is at most a few dozen, so proofs are <100 bytes.

### 8.5 Protocol parameter table

| Parameter | Default | Range | Comment |
|---|---|---|---|
| Lattice dimension (number of primes used as axes) | 16 | 8–32 | More axes = finer slabs |
| N (cyclotomic ring dimension) | 256 | 128–1024 | Tied to attention head dim |
| k (CRT primes) | 2 | 2–8 | Trade latency for resilience |

| Parameter | Default | Range | Comment |
|---|---|---|---|
| q_i bits | 30 | 30 | Fixed by NTT prime catalog |
| Gossip round (s) | 30 | 10–120 | Tunable per node class |
| Epoch length (s) | 600 | 60–3600 | Frontier publication cadence |
| Replication factor r | 3 | 2–8 | Slab over-coverage |
| Verification sample rate | 1/1024 | 1/128 – 1/65536 | Per-layer recomputation |
| KSTE commitment size | 64 B | fixed | On wire |
| HRR delta size | ~4 KB | per gossip round per neighbor | |
| CRT residue per layer | ~8 KB | per inference step per shard | |

---

## 9. Bootstrap, epochs, and governance

### 9.1 New-node bootstrap

A new node joins by:

1. Generating an Ed25519 identity keypair.
2. Connecting to a published seed node (the seed list is bootstrapped out-of-band, exactly as Bitcoin and Kademlia do).
3. Announcing a candidate slab (a bounded region of L) it wants to own.
4. The seed node and its lattice neighbors negotiate the actual slab assignment by consulting the current coverage map: regions under-replicated relative to r are preferred; over-replicated regions are refused.
5. Once assigned, the new node downloads the dominance frontier for its slab from the existing holders (this is bounded — Section 2.3 guarantees the frontier is finite per slab).
6. The new node begins crawling, binding, and gossiping.

The bootstrap window for a small slab is on the order of minutes; for a large slab it can be hours, dominated by frontier download time.

### 9.2 Epoch structure

The network advances in epochs of (default) 600 s. At each epoch boundary:

- The dominance frontier for every slab is finalized and published (its hash is the **epoch root**).
- New bindings accepted during the epoch are folded into the next epoch's HRR base state.
- Inference verification challenges issued during the epoch are settled.
- WRK and DSC are minted for verified contributions.
- Stake slashing is applied for caught misbehavior.

Epoch roots form a chain (each root commits to the previous root), giving the network a lightweight ledger for auditing. The chain is **not** a consensus blockchain — there is no global ordering of every microtransaction. It is a snapshot ledger of the dominance frontier and the cumulative mint state.

### 9.3 Slashing conditions

Stake (denominated in DSC) is slashed when:

1. **Forged commitment**: a published KSTE commitment fails $\preceq_d$ verification against the alleged content. Slash: full stake at the offending coordinate.
2. **Divergent shard**: an inference shard's residue disagrees with the verification recomputation. Slash: full stake at the offending shard, plus return of fees.
3. **Sybil binding**: a node submits HRR bindings that are detectably correlated with another identity's bindings (same key-derivation seed, same gradient pattern). Slash: full stake on all detected identities.
4. **Equivocation**: a node publishes two different commitments for the same coordinate in the same epoch. Slash: full stake.

Slashing is performed by the next-epoch validator quorum (the set of DSC holders who have staked into the validator role for that epoch); proofs are public so slashes are independently verifiable.

### 9.4 Governance

DSC holders vote on protocol parameters (Section 8.5), upgrade proposals, and reserve-fund allocation. Vote weight is the geometric mean of (DSC held, length of holding, frontier coordinates the holder has contributed to). The last term prevents pure-financial capture: a holder who has not contributed to the frontier has bounded governance weight no matter how much DSC they accumulate.

Upgrade activation uses a soft-fork mechanism: a proposed change is signaled by a fraction of stake; activation occurs at a future epoch boundary once a threshold (default 67%) is reached.

---

## 10. Failure modes and adversarial considerations

We treat each adversarial mode with its own paragraph and an honest assessment of what is and is not covered.

### 10.1 Sybil attacks

The standard Sybil concern is that an attacker spawns many cheap identities and overwhelms the network. In our system, $\preceq_d$ verification is **identity-blind** — it does not matter whether a contribution comes from one identity or a thousand; only whether the tree embeds correctly into the frontier. The discovery token mints **only** for genuine novelty, so a Sybil swarm submitting redundant contributions earns nothing in DSC. WRK is earned per verified compute unit, so a Sybil swarm with no real compute earns nothing in WRK either. The remaining Sybil vector is **stake dilution in governance**: a swarm that has somehow acquired DSC can vote with many identities. We mitigate by tying governance weight to frontier contributions (§9.4), but this is partial; large DSC accumulations remain influential, and we treat this as an open problem in §12.

### 10.2 Data poisoning

A poisoner submits a KSTE-encoded commitment that is malformed or adversarial in content (e.g., a tree that encodes nothing semantically useful but happens to be dominance-novel). Because KSTE encoding is **deterministic** — the same content produces the same tree — anyone can re-encode the source content and check that the published tree matches. Mismatched

commitments are slashable (§9.3). The deeper attack is content that is genuinely on the web but harmful (spam, propaganda); this is a content-moderation problem we do not solve algorithmically. The network's role is to faithfully represent what is on the web; downstream consumers (training pipelines, inference users) apply their own content filters.

### 10.3 Eclipse and Byzantine routing

A standard Kademlia weakness is that an adversary surrounding a node in the overlay can eclipse it from honest peers. Our lattice geometry helps: a node's neighbors are arithmetically determined by its coordinate, not by a peer-discovery handshake. An eclipser would need to control the actual coordinates adjacent to the target, which requires (a) joining slabs in those coordinates, and (b) being assigned to them by the bootstrap negotiation, which prefers under-replicated coordinates. This does not eliminate the attack but raises its cost substantially. We additionally recommend the standard Kademlia defenses (S/Kademlia node-ID hashing, redundant lookup paths).

### 10.4 Free-rider problem

A node that consumes inference but contributes no compute is a free-rider. WRK fees on inference requests directly compensate: a free-rider must pay WRK, which they can only acquire by either contributing compute themselves or buying WRK on the market (which transfers value to contributors). The system does not require altruism; it requires that the WRK market clears. The economic risk is that WRK price collapses (Section 7.5); we treat this as open.

### 10.5 Capacity exhaustion in ARM under heavy aggregation

When a slab receives more bindings than the local ARM ring can hold without catastrophic capacity loss, the slab triggers **ring decomposition** (§3.3): it splits into two sub-slabs, each with a sub-ring `Z_q[x]/(x^{N/2}+1)`, and partitions coefficients between them. Sub-slab assignment uses lattice geometry: existing bindings are redistributed to whichever sub-slab their lattice coordinate now belongs to. The cost is a one-time epoch boundary at which all affected nodes participate; the benefit is doubled capacity for the affected region. Repeated decomposition gives exponential headroom but is rarely triggered in practice because gossip plus capacity decay (§3.3) absorbs most aggregation pressure.

### 10.6 Stale gradients in distributed training

In synchronous SGD, stale gradients hurt convergence. In our setting, ARM's automatic capacity decay (§3.3) acts as an implicit staleness filter: old bindings fade as new ones accumulate, so a stale gradient that arrives late has its influence already attenuated. We do not promise convergence rates competitive with synchronous SGD on a well-coordinated cluster — that is the regime where Hivemind, DiLoCo, and friends already shine — but we do promise that the system **does not fail catastrophically** under stale gradients, which is the failure mode of naive averaging schemes.

### 10.7 Network partitions

If the network partitions into two halves, each half continues to operate on its own dominance frontier. CRT-sharded inference degrades to working with the subset of primes whose shards are on the same side of the partition; this lowers the working modulus but does not stop inference. When the partition heals, the two frontiers are merged: dominance-comparable entries are deduplicated, dominance-incomparable entries from both sides are unioned (the wqo property guarantees the merged frontier remains finite). HRR states are summed across the partition boundary; commutativity ensures correctness.

### 10.8 Verifier collusion

The verification samples (§4.3) require honest validators. A colluding ring of validators could skip checks. We mitigate by (a) making validator selection random per epoch (weighted by DSC stake), (b) rotating validators, and (c) allowing any node to challenge any decision retroactively within a slashing window. A colluding majority of DSC stake can break the system; this is the standard PoS attack and we do not claim to have a novel defense against it.

---

## 11. Comparison to prior work

**Hivemind / DiLoCo**: distributed training with carefully managed gradient synchronization. They optimize the gradient aggregation step and accept fully centralized data and model coordination. We aggregate via ARM (no explicit synchronization), and our crawl/inference layers are also decentralized. They are likely faster to train a single model on a coordinated cluster; we are likely more robust under churn and adversarial conditions.

**Petals**: BLOOM-style model parallelism over volunteer GPUs. Slices the model layer-wise across nodes and routes activations through the network. We slice via CRT residues (algebraic, not sequential) so per-layer wire traffic is kilobytes and verification is exact. Petals' wire traffic is megabytes per layer and verification is approximate. Petals serves a single shared model; we host many models concurrently because the lattice naturally separates them.

**Bittensor**: incentive-driven decentralized ML with a single token (TAO) and an emission-based reward. We have two tokens with deliberately different inflation curves to avoid the single-token collapse pattern (§7.5). Bittensor verifies via subjective stake-weighted scoring; we verify via objective $\preceq_d$ embedding, which is mathematically stronger and removes the validator-as-oracle problem.

**Filecoin**: decentralized storage with proof-of-replication and proof-of-spacetime. Our verification is closer in spirit to Filecoin than to Bitcoin — both rely on cryptographic commitments to data the prover holds. We do not need spacetime proofs because the network's value lives in the dominance frontier, not in raw storage; nodes can re-derive content from any replica.

**BitTorrent**: content distribution via a swarm. We share the swarm property (no central coordinator, content located by hash) but our hash is the lattice coordinate, which carries semantic structure. Swarms in BitTorrent self-organize per torrent; our swarm self-organizes globally by topic.

**YaCy**: decentralized web search. Closest in spirit to our Layer 1, but YaCy uses a uniform DHT and a flat keyword index. We use the lattice DHT and a dominance-pruned KSTE cache; the result is sub-linear cache growth and direct semantic adjacency in routing.

**Folding@home / BOINC**: volunteer scientific compute. Workload assignment is centralized (the server hands out tasks). We assign work via lattice geometry with no central scheduler, and our work is **continuously useful** (crawl, train, infer) rather than per-task batched.

---

## 12. What this paper does NOT establish

We are honest about the gaps. The companion theory paper (PPT-LAT-T) establishes the math: lattice geometry, $\preceq_d$ decidability, CRT NTT correctness, ARM capacity bounds. This systems paper specifies an architecture built on that math, but several things are not established:

1. **Scaling constants**. We claim O(log N) routing, O(neighborhood) admit decisions, kilobyte-scale per-layer residues. The constants in front of these asymptotics matter

enormously at the scale of a real volunteer network. We do not have measured constants; we have engineering estimates calibrated against the existing shannon-prime-engine codebase, which is single-node.

2. **Economic equilibria**. The two-token design (§7) is argued informally. We have not modeled the supply/demand curves of WRK and DSC under adversarial trading, governance attacks, or coordinated abandonment. Standard mechanism-design analysis is needed before we can claim the design avoids the collapse modes of prior compute coins.
3. **Convergence of distributed training under ARM**. We argue that capacity decay handles staleness; we do not prove convergence on any specific learning objective. Empirical studies on a deployed network are required.
4. **Quality of lattice-coordinate semantic adjacency**. We claim URLs hash to coordinates so that adjacency is semantic. This is a property of the chosen hash function (Section 2.1), and we have not benchmarked alternative hash designs against, say, embedding-cosine similarity at scale.
5. **Robustness of slab assignment under churn**. The slab negotiation procedure (§9.1) handles steady-state joins and leaves. Adversarial churn — coordinated mass-join, coordinated mass-leave — is not analyzed.
6. **Censorship resistance vs. content moderation tradeoff**. The network faithfully reproduces what is on the web. We do not address jurisdictional content-removal requirements; that is a deployment question, not an algorithmic one.
7. **Real-world bootstrap cost**. The frontier-download step in bootstrap (§9.1) is bounded in principle by the wqo, but the bound's practical value depends on the prime alphabet and the model's vocabulary. We have not measured this.
8. **Interaction with model-architecture changes**. When a model architecture changes (new modality, new tokenizer), the lattice coordinate scheme may need to extend. Our claim is that the lattice is extensible; we have not specified the migration protocol.

The top three of these — scaling constants, economic equilibria, and convergence of distributed training under ARM — are the questions that any implementation effort will need to confront first. We treat them not as flaws but as the work that comes next.

---

*PPT-LAT-S, draft. Companion to PPT-LAT-T (theory) and PPT-LAT-X (experiments, forthcoming).*